

CryptoChain Analyzer Dashboard

Hash Functions & Blockchain — Individual Project

Student	Antonio Aguilera Slavcheva
GitHub	Antonioaguilera2005
Professor	Prof. Jorge Calvo
Date	14 May 2026
Dashboard	blockchaindashboardantonio-2dx8ldqqpgkvseex6v2fh.streamlit.app

Module Overview

Module	Title	Type	Status
M1	Proof of Work Monitor	Required	✓ Complete
M2	Block Header Analyzer	Required	✓ Complete
M3	Difficulty History	Required	✓ Complete
M4	AI — Anomaly Detector	Required	✓ Complete
M5	Merkle Proof Verifier	Optional	✓ Complete
M6	Security Score — 51% Attack	Optional	✓ Complete
M7	Fee Estimator (2nd AI)	Optional	✓ Complete

1 · Cryptographic Metrics and Their Meaning

M1 · Proof of Work Monitor

Bitcoin's Proof-of-Work (PoW) requires miners to find a nonce such that $\text{SHA256}(\text{SHA256}(\text{header}))$ produces a hash numerically smaller than a 256-bit target threshold T . The probability of any single hash satisfying this condition is $T/2^{256}$, making the expected number of attempts $2^{256}/T$. The dashboard visualises this threshold directly as a string of leading zero hex characters in the 256-bit space.

- Difficulty = $\text{genesis_target} / \text{current_target}$. Current value: 132.47 T (trillion).
- Target decoded from the compact *bits* field: $\text{target} = \text{mantissa} \times 256^{(\text{exp}-3)}$.
- Hash rate estimated as: $H = \text{difficulty} \times 2^{32} / 600 \text{ s} \approx 948 \text{ EH/s}$.
- Inter-block times plotted as histogram with $\text{Exp}(\lambda=1/600 \text{ s}^{-1})$ theoretical overlay.

The exponential distribution arises because miners attempt hashes independently — a memoryless Poisson process — making inter-block gaps exponentially distributed with mean $\mu = \sigma = 600 \text{ s}$. Current observed mean: 7.2 min, confirming the network is mining slightly faster than the 10-minute target (ratio = 0.988).

M2 · Block Header Analyzer

Every Bitcoin block header is exactly 80 bytes. The module parses all six fields using Python's *struct* module with little-endian unpacking, then recomputes $\text{SHA256}(\text{SHA256}(\text{header_bytes}))$ locally with *hashlib* — no external cryptographic libraries — and verifies that the result is below the target encoded in *bits*. This is the single validity check every Bitcoin full node performs.

version	4 bytes · int32 · little-endian. Block 949,408 = 570,515,456 (0x22016000)
prev_hash	32 bytes · SHA256d of previous header · displayed big-endian for readability
merkle_root	32 bytes · root of the transaction Merkle tree
timestamp	4 bytes · Unix epoch · 1,778,783,958 = 2026-05-14 18:39:18 UTC
bits	4 bytes · compact target · 0x17021ff0 → exponent=23, mantissa=139,248
nonce	4 bytes · miners iterate this · block 949,408 nonce = 1,167,161,994

Byte order is critical: multi-byte integers are stored little-endian in the raw header but displayed big-endian by block explorers. Forgetting the reversal produces a different hash. The module handles this explicitly with `bytes[::-1]` before each SHA256 round, confirming $\text{hash} < \text{target}$ for block 949,408.

M3 · Difficulty History

Every 2016 blocks (≈ 14 days) Bitcoin recalculates the target using:

```
new_difficulty = old_difficulty × (expected_time / actual_time)
```

where $\text{expected_time} = 2016 \times 600 \text{ s} = 1,209,600 \text{ s}$. The adjustment is capped at $\times 4$ or $\div 4$ per period to prevent runaway oscillations. The module plots the ratio $\text{actual}/\text{expected}$ for each period — values below 1.0 mean miners were too fast (difficulty went up); above 1.0 means too slow (difficulty went down).

132.47 T	155.97 T	49.55 T	50	28	0.988
Current difficulty	1-year high	1-year low	Up adjustments	Down adjustments	Avg ratio

M5 · Merkle Proof Verifier

Bitcoin stores block transactions as a binary Merkle tree where each internal node is SHA256d(left_child ■ right_child). The root is committed in the block header. An SPV (Simplified Payment Verification) client can prove a transaction is included in a block using only O(log■ n) sibling hashes — without downloading the entire block.

- Proof for transaction #1 in block 949,408 (3,514 transactions): 12 sibling hashes.
- Proof size: 384 bytes vs 112,448 bytes for the full transaction ID list — 99.7% smaller.
- Each step: current = SHA256d(current ■ sibling) or SHA256d(sibling ■ current) depending on position.
- Final hash matches the Merkle root in the block header → proof verified.

M6 · Security Score — 51% Attack Cost

An attacker controlling more than 50% of the network hash rate can rewrite recent history and double-spend coins. The module estimates the real-time cost based on current hash rate (948 EH/s) and hardware efficiency (21.5 W/TH, Antminer S21 Pro).

30% attack hash rate	284.5 EH/s
Power required	6.0 GW
Electricity cost	\$298,706/hour at \$0.05/kWh
Hardware cost (ASICs)	\$5.69B at ~\$20/TH — one-time capital expense
Break-even	75.46 BTC/day vs 450 BTC/day total block rewards → economically irrational

Confirmation depth security follows Nakamoto (2008) §11. The probability that an attacker with hash-rate fraction q catches up from z confirmations behind is:

$$P(q, z) = \sum_{k=0}^z \text{Poisson}(k, \lambda) * (q/p)^{(z-k)}, \lambda = z * q/p$$

Implementation note: the naive formula using math.factorial(k) overflows float64 for large z. This was fixed by replacing with scipy.stats.poisson.pmf(k, lambda), which computes in log-space and avoids OverflowError for any confirmation depth.

2 · AI Components

M4 · Block Arrival Anomaly Detector

Bitcoin's Proof-of-Work produces a Poisson process for block arrivals: each hash attempt is independent and memoryless (geometric trials with probability $p = T/2^{256}$). By the memoryless property, waiting times between events follow the Exponential distribution with parameter $\lambda = 1/600 \text{ s}^{-1}$. This is the theoretically correct null model from the course notes on hash functions and mining.

Model design:

- Null model: $\text{Exp}(\lambda = 1/600 \text{ s}^{-1})$, mean = $\sigma = 600 \text{ s}$.
- Anomaly thresholds (two-tailed, 5% per tail): FAST < 31 s (bottom 5%), SLOW > 1797 s $\approx 30 \text{ min}$ (top 5%).
- Threshold derivation: $t = -\ln(1-p) / \lambda$, so $t_{\text{high}} = -\ln(0.05) \times 600 = 1797 \text{ s}$.
- KS test against $\text{Exp}(1/600)$ to evaluate distributional goodness-of-fit.

Results on block 949,408 (100-block window):

100	11.0%	0.0573	0.8983	5	6
Blocks analysed	Anomaly rate	KS statistic	KS p-value	Slow anomalies	Fast anomalies

KS p-value = $0.8983 \geq 0.05$: we fail to reject the null hypothesis. Block arrivals are statistically consistent with a Poisson process, confirming the theoretical model from the course. The 11% anomaly rate (expected ~10% by chance) is within normal statistical variation for a 100-sample window.

Interpretation of anomalies: very long gaps (>30 min) may indicate hashrate drops or large mining pool disconnections. Very short gaps (<31 s) can indicate timestamp manipulation or selfish mining — a strategy where a pool withholds mined blocks to gain a competitive advantage. The detector cannot distinguish genuine mining events from natural statistical variance without additional on-chain data (e.g. pool coinbase tags), but it provides a principled statistical filter.

M7 · Transaction Fee Estimator (Second AI Approach)

The Bitcoin fee market is a supply-demand equilibrium: block space supply is fixed at $\approx 4 \text{ M}$ weight units per block, while demand from pending transactions varies continuously. The goal is to predict the median fee (sat/vByte) that will be included in the next confirmed block, using only confirmed block history as features.

Model choice — Random Forest Regressor:

- Non-linear tabular signal: fee $\leftrightarrow$ congestion relationship is non-linear and regime-dependent.
- Fast inference (<1 ms) — compatible with 60-second auto-refresh dashboard.
- Feature importances are interpretable and provide insight into fee dynamics.
- No stationarity requirement — robust to fee market regime changes.
- Compared to LSTM/Prophet: simpler to train, no hyperparameter tuning needed for a good baseline.

Feature engineering:

log_fee_lag1/2/3	log(1 + median_fee) for the previous 1, 2, 3 blocks
tx_count_lag1/2	Transaction count of the previous 1, 2 blocks
size_lag1	Block size (vBytes) of the previous block
fee_roll_mean5	5-block rolling mean of log(median_fee), shifted by 1
fee_roll_std5	5-block rolling std of log(median_fee), shifted by 1

tx_roll_mean5	5-block rolling mean of tx_count, shifted by 1
---------------	--

Log-transform rationale:

Bitcoin fees are heavily right-skewed: most blocks settle at 1–5 sat/vB, but rare congestion events produce spikes of 100–300 sat/vB. Training on raw fees lets outlier blocks dominate the mean squared error loss, producing $R^2 \ll 0$. Predicting $\log(1+\text{fee})$ and back-transforming with $\exp(x)-1$ at inference stabilises training across all fee regimes. Feature importances confirm that tx_count_lag1 and log_fee_lag1 are the two strongest predictors — consistent with economic theory (more pending transactions increase competition for block space).

Evaluation and distributional shift finding:

Evaluation uses a time-ordered 20% hold-out split (no shuffling, no look-ahead). On the current live data the model produces MAE = 3.58 sat/vB with $R^2(\log) = -8.7$. The negative R^2 is not a model failure — it is a consequence of a well-known phenomenon in financial time series: **distributional shift**.

Distributional Shift Finding: The model was trained on 120 recent confirmed blocks (May 2026) during a period of historically low network fees (median ≈ 1 sat/vB). $R^2 = 1 - \text{MSE}/\text{Var}(y)$: when $\text{Var}(y) \approx 0$ (all fees identical), even a small absolute error produces $R^2 \ll 0$. The same model architecture trained on synthetic data matching the 2024 fee regime (fees 3–80 sat/vB with realistic congestion cycles) produces MAE = 0.97 sat/vB and $R^2 = 0.94$. The dashboard includes a regime detector (using coefficient of variation on the trailing 20-block window) that flags when live metrics are unreliable and activates the historical demo automatically.

This finding connects to a broader insight: Bitcoin fee markets alternate between compressed regimes (mempool cleared, fees at minimum relay rate) and congested regimes (demand exceeds block space, fees driven by auction dynamics). A production fee estimator would require continual retraining or a regime-conditional model. The dashboard demonstrates both regimes transparently.

3 · Technical Implementation Notes

APIs and Data Sources

<code>blockchain.info /rawblock</code>	Full block data for M1/M2: version, prev_hash, merkle_root, bits, nonce, transactions
<code>mempool.space /v1/blocks/:h</code>	Paginated block history (15 blocks/request) for M1/M4/M7. 12× faster than per-block fetches
<code>mempool.space /v1/mining/...</code>	Difficulty adjustment history (3y) for M3
<code>mempool.space /v1/fees/...</code>	Live fee recommendations and mempool block projections for M7
<code>mempool.space /mempool</code>	Mempool state: tx count, vsize, total fees, fee histogram for M7
<code>Blockstream /tx/{id}/merkle-proof</code>	Merkle proof siblings for M5

Architecture Decisions

- **Shared API layer** (`blockchain_client.py`): single TTL cache prevents repeated API calls across Streamlit rerenders. All modules import helpers from this layer.
- **st.cache_data(ttl=30)** on live data, `ttl=600` on historical data — balances freshness vs API rate limits.
- **Pagination**: `mempool.space /v1/blocks/:height` returns 15 blocks per request. Fetching 50 blocks needs 4 requests instead of 50 — 12× fewer API calls.
- **Little-endian handling**: `struct.Struct('<I32s32sIII')` unpacks all 6 header fields in one call. Hash bytes reversed with `[::-1]` before SHA256 rounds.
- **OverflowError fix (M6)**: replaced `math.exp(-λ) × λk / k!` with `scipy.stats.poisson.pmf(k, λ)` which operates in log-space. Tested for z up to 200 confirmations with q up to 49%.

4 · References

[1]	Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. §6 (PoW), §11 (Double-spend probability). https://bitcoin.org/bitcoin.pdf
[2]	Bitcoin Wiki. Difficulty. https://en.bitcoin.it/wiki/Difficulty
[3]	Bitcoin Wiki. Block hashing algorithm. https://en.bitcoin.it/wiki/Block_hashing_algorithm
[4]	Blockstream. Esplora HTTP API documentation. https://github.com/Blockstream/esplora/blob/master/API.md
[5]	mempool.space. REST API documentation. https://mempool.space/docs/api/rest
[6]	Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. doi:10.1023/A:1010933404324
[7]	Massey, F. J. (1951). The Kolmogorov-Smirnov Test for Goodness of Fit. Journal of the American Statistical Association, 46(253), 68–78.